

MSc Creative Computing

MSc Creative Computing

Practice as Research — Lecture 5

Reflection and documentation — turning process into evidence

Mick Grierson — Creative Computing Institute, UAL

Black text on white. ~20 slides. Speaker notes under each.

Practice as Research

Reflection and Documentation — Turning Process into Evidence

Lecture 5 of 8 — Practice as Research

Mick Grierson, [Creative Computing Institute](#), UAL

Recap: where we are

Week 1: Research = edge + hypothesis + test.

Week 2: Practice produces knowledge — including tacit and experiential knowledge.

Week 3: Finding the gap.

Week 4: Choosing and designing methods.

Today: **How do you capture what you learn through making — and turn it into research evidence?**

Two connected problems:

1. **Reflection** — how do you notice and analyse what's happening in your own practice?
2. **Documentation** — how do you record it so that it becomes communicable to others?

Why reflection matters

In Lecture 2 we said practice produces **tacit** and **experiential** knowledge — knowledge that resists full verbalisation.

The problem: if it resists verbalisation, how do you include it in a research submission?

The answer is not "make it fully propositional." Some knowledge will remain partially tacit — and that's fine. The answer is: **reflect on your practice carefully enough that you can articulate as much as possible, and point to the artefact for the rest.**

Reflection is the bridge between doing and knowing. It is what turns "I made a thing" into "here is what making the thing revealed."

Schön: the reflective practitioner

Donald Schön (1983) identified two modes of reflection that professionals use:

Reflection-in-action — thinking *while* you're doing. The moment you hear your generative system produce an unexpected timbre and you adjust the parameters in response — that's reflection-in-action. You're reading the situation and responding to it in real time.

Reflection-on-action — thinking *after* you've done something. You step back from the session, look at what happened, and ask: what worked, what didn't, what surprised me, what does this mean for the next iteration?

Both are research activities, but only the second one reliably produces *records*. The challenge is capturing reflection-in-action before it fades — because in-the-moment insights are often the most valuable and the most perishable.

Reflection is not description

There is a crucial difference between describing what you did and reflecting on it.

Description: "I trained the model for 200 epochs on the vocal dataset."

Reflection: "I trained the model for 200 epochs on the vocal dataset. At epoch 80, the outputs started producing recognisable vowel shapes but with metallic artefacts. I initially treated this as a training failure, but on listening more carefully I realised the artefacts had a timbral character that was musically interesting — closer to a processed vocal than a clean one. This changed my framing: instead of aiming for high-fidelity vocal reproduction, I began exploring what the model's 'errors' offered as a distinct sonic palette. The gap in my project shifted from 'can neural networks reproduce voice?' to 'what happens when they fail to, and is that failure aesthetically productive?'"

The first is a log entry. The second is a research insight.

A framework for reflective notes

When you sit down to reflect on a session of making, ask yourself four questions:

What happened? — Describe the activity. What did you do, build, test, change?

What surprised you? — Surprises are where learning lives. Expected outcomes confirm what you already knew. Unexpected outcomes teach you something new.

What does it mean? — Interpret the surprise. Why did it happen? What does it reveal about your system, your assumptions, your question?

What changes? — How does this affect what you do next? Does it refine your question? Alter your method? Open a new direction?

This cycle — do, notice, interpret, adjust — is the iterative cyclic web from Lecture 2, operating at the scale of a single work session.

Why documentation matters

Documentation serves three audiences:

Your future self — in three months you will not remember why you made that design decision, what that parameter value was, or what the output sounded like before you changed the architecture. Your log is your memory.

Your examiner — they weren't in the room. They didn't hear the output, see the failure, or feel the surprise. Your documentation is their only access to the parts of your research that the final artefact alone doesn't convey.

The field — if your work is good, other people will want to build on it. They need to know what you did, how you did it, and what you learned. Reproducibility and extendability depend on documentation.

Audiences

What to document: the five layers

Think of documentation as operating at five layers, from the most granular to the most interpretive:

1. The artefact itself — the system, the code, the outputs, the installation, the performance. This is the primary evidence.

2. Technical specification — architecture, dependencies, parameters, training data, hardware. Enough detail that someone could, in principle, reproduce or extend your work.

3. Process record — what you did, in what order, with what results. Dated, sequential, factual. This is your lab notebook.

4. Reflective commentary — the four-question framework from slide 6. What surprised you, what it meant, what changed. This is where tacit knowledge becomes partially explicit.

5. Critical framing — how the artefact and the process relate to the research question, the gap, and the field. This is the interpretive layer that turns documentation into argument.

Layers 1–3 are **records**. Layer 4 is **reflection**. Layer 5 is **analysis**. You need all five, but they serve different purposes and often live in different places.

[Five Layers](#)

Where documentation lives

Different layers live in different places. Here's a practical breakdown:

Code and system: GitHub or GitLab. Use meaningful commit messages — they are process documentation for free. Tag significant versions. Write a proper README.

Process log: A regular document — Markdown, Notion, a physical notebook, whatever you'll actually use. Date every entry. Aim for at least one entry per working session.

Audio/visual outputs: Timestamped exports. Keep a folder structure that makes sense. Name files meaningfully — `output\_2025-03-14\_spectral-loss-v2.wav` not `test7\_final\_FINAL.wav`.

Reflective notes: In the process log, or as a separate layer on top of it. Some people prefer a separate reflective journal; others annotate their process log.

Video and screen recordings: For installations, performances, interactive systems, and anything where the experience is temporal. OBS is free. Record more than you think you need.

The written submission: Where layers 4 and 5 ultimately live in their polished form.

The annotated portfolio

Gaver & Bowers (2012) proposed the **annotated portfolio** as a method for presenting design artefacts as research contributions.

The idea: you present a curated selection of artefacts (prototypes, outputs, iterations, experiments) and accompany each with written annotation that explains **what it shows, why it matters, and how it relates to the others**.

The portfolio is not a gallery. It is an **argument made through artefacts**.

Each annotation should address:

- What is this artefact? (What was it made with, when, in what context?)
- What does it demonstrate? (What knowledge does it carry?)
- What design decisions does it embody? (What choices were made, and why?)
- How does it relate to the other items in the portfolio? (What trajectory or development does the collection reveal?)

An annotated portfolio in practice

Imagine you're building a system that generates visual patterns from audio input.

Item 1: Early prototype. Direct amplitude-to-brightness mapping.

Annotation: "The first mapping was literal — loud = bright, quiet = dark. The result was visually responsive but uninteresting. It confirmed that direct parameter mapping produces correlation without aesthetic surprise."

Item 2: Second prototype. Spectral centroid mapped to colour hue, onset density mapped to pattern complexity.

Annotation: "By mapping perceptual audio features to visual properties, the system began producing outputs that 'felt' connected to the sound rather than merely synchronised with it. The distinction between synchronisation and connection became a central concept in the project."

Item 3: Final system. Multi-feature mapping with a learned crossmodal embedding.

Annotation: "The learned embedding produced unexpected cross-modal associations — certain timbres consistently evoked angular shapes, which aligns with research on sound symbolism (Köhler, 1929; Ramachandran & Hubbard, 2001). The system's outputs became evidence for a relationship between audio and visual features that was discovered through making, not predicted by theory."

Three items. Three annotations. A clear trajectory from naive mapping to discovered insight.

Documenting failure

Failures, dead ends, and abandoned directions are **research evidence**, not embarrassments.

If you tried an approach and it didn't work, that's valuable information — provided you document *why* it didn't work and *what you learned from it*.

A project that presents only its final, successful outcome looks less rigorous than one that shows: "I tried A, B, and C. A failed because of X. B was partially successful but had limitation Y. C, informed by what I learned from A and B, addressed both X and Y."

The narrative of informed iteration is one of the strongest things you can present in a practice-based submission.

Documenting for different audiences

Not all documentation goes in the final submission. Think of it as concentric rings:

Inner ring — for you: The raw process log. Everything. Messy, dated, honest. This is your working memory. It stays in your notebook or your private repo.

Middle ring — for the examiner: The curated evidence. Selected log entries, annotated portfolio items, key outputs, the reflective commentary that supports your argument. This goes in the appendices or the body of your submission.

Outer ring — for the field: The reproducibility package. Code on GitHub with a README, trained models if shareable, a clear description of the method. This is what lets others build on your work.

You write the inner ring first and extract the other two from it. If the inner ring doesn't exist, there's nothing to extract.

What "good enough" documentation looks like

You don't need a cinematic production diary. You need:

Process log: One entry per significant working session. 100–300 words. Uses the four-question framework (what happened, what surprised you, what does it mean, what changes). Dated.

Version-controlled code: With meaningful commit messages. Tagged at significant milestones. A README that explains how to run it, what it does, and what it depends on.

Timestamped outputs: Audio, images, video — whatever your system produces. Named sensibly. Kept in chronological order so you can trace the development.

At least one screen recording or demo video of your system in action. Even if it's rough. The examiner needs to see the thing working, not just read about it.

Reflective notes at key decision points: when you changed direction, when something failed, when something surprised you, when you made a significant design choice.

This is not a huge overhead. It's 15 minutes of writing per session, plus basic file hygiene.

Documentation and the iterative cyclic web

Smith & Dean's iterative cyclic web (Lecture 2) describes a process of making, reflecting, reading, and revising — repeatedly.

Documentation is how you **make that cycle visible**. Without it, the cycle happens but nobody (including you) can trace it afterwards.

Each pass through the cycle should leave a trace:

- **Make** → outputs, code commits, prototypes.
- **Reflect** → log entries, reflective notes.
- **Read** → annotated references, notes on how the reading changed your thinking.
- **Revise** → updated question, changed parameters, new direction — documented with a note explaining *why* you revised.

If your documentation lets someone reconstruct the trajectory of your thinking — from first hunch to final artefact, through all the surprises and pivots — you have a strong evidential foundation for your submission.

A worked example: documenting a project in creative computing

Here's what good documentation might look like across the life of a 12-week MSc project:

Weeks 1–3: Process log established. Research question drafted. Literature notes with five-point extraction (from Lecture 3). Gap statement drafted. Early sketches and architectural diagrams.

Weeks 4–6: First prototype built. Code on GitHub. Initial outputs exported and timestamped. Log entries note first surprises and pivots. Reflective notes at key decision points ("chose spectral loss over waveform loss because...").

Weeks 7–9: Second iteration built, informed by reflections on the first. User study conducted (if applicable). Screen recordings and demo videos captured. Annotated portfolio beginning to take shape — three or four items with annotations.

Weeks 10–12: Final system documented. Process log reviewed and key entries extracted for the written submission. Annotated portfolio finalised. Reflective commentary written up as part of the methods and discussion sections. README polished. Code tagged as final version.

Common mistakes

"I'll document later." You won't. Or you will, but you'll forget the insights that mattered most. Document as you go.

"My code is my documentation." Code is layer 2 (technical specification). It doesn't capture why you made the choices you did, what surprised you, or how your understanding changed. You need layers 3, 4, and 5 as well.

"I documented everything." If you documented *everything* equally, you documented nothing usefully. Documentation for the examiner is curated — you select the moments that matter and explain why they matter.

"The artefact speaks for itself." It doesn't — not to someone who wasn't in the room while you made it. The artefact speaks *with* the documentation. Alone, it's mute.

"Reflection means saying what I felt." Reflection is analytical, not emotional. "I felt frustrated" is not a research insight. "The system consistently failed at long sustained notes, which frustrated me, but which also revealed that the architecture handles transients better than steady-state signals — a finding I hadn't anticipated" — that's a research insight.

An exercise: write a reflective note

Think about the last significant thing you worked on — for this module, for another project, or in your own practice.

Write a short reflective note (150–200 words) using the four-question framework:

1. **What happened?**
2. **What surprised you?**
3. **What does it mean?**
4. **What changes?**

Take 10 minutes. We'll hear a couple of examples.

For next week

Week 7: **Exhibition vs research output — complementary, not opposed.**

Your project will likely be exhibited or demonstrated in some form. Next week we ask: how does exhibition relate to the research submission? Are they the same thing? Different things? How do you design for both?

Between now and then:

- Start (or continue) a process log. One entry per working session.
- Export and timestamp your outputs.
- Write at least one reflective note using the four-question framework.

References

Schön, D. (1983). *The Reflective Practitioner: How Professionals Think in Action*. Basic Books.

Gaver, W. & Bowers, J. (2012). ["Annotated Portfolios."](#) *Interactions*, 19(4), 40–49.

Smith, H. & Dean, R. T. (eds.) (2009). [*Practice-led Research, Research-led Practice in the Creative Arts.*](#) Edinburgh University Press.

Borgdorff, H. (2011). "The Production of Knowledge in Artistic Research." In Biggs & Karlsson (eds.), [*The Routledge Companion to Research in the Arts.*](#) Routledge, pp. 44–63.

Nelson, R. (2013). [*Practice as Research in the Arts.*](#) Palgrave Macmillan.

Kolb, D. (1984). *Experiential Learning: Experience as the Source of Learning and Development*. Prentice-Hall.

Köhler, W. (1929). *Gestalt Psychology*. Liveright.

Ramachandran, V. S. & Hubbard, E. M. (2001). ["Synaesthesia — A Window into Perception, Thought and Language."](#) *Journal of Consciousness Studies*, 8(12), 3–34.

Questions

Have you started a process log?

What's the hardest thing to capture about your own practice?

What are you afraid of documenting?